

PROJECT REPORT

MERN Stack Employee Management System

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Application

Full Stack Development

STUDENT NAME

Bhanu Sharma

ENROLLMENT NO.

O24MCA112292

GUIDE / MENTOR

Alok Srivastva

ACADEMIC YEAR

2024-2026

UNIVERSITY

Chandigarh University

REPOSITORY

<https://github.com/nextlvlpro/mern-employee-app>

Certificate

This is to certify that the project report titled **MERN Stack Employee Management System** has been prepared and submitted by **Bhanu Sharma**, Enrollment No. **O24MCA112292**, in partial fulfillment of the requirements for the award of the degree of Master of Computer Application.

The project work has been carried out under the guidance of **Alok Srivastva** during the academic year **2024-2026**.

Copy of the certificate received from Qollabb or the concerned authority may be pasted here if required by the submission portal.

Guide / Mentor Signature
Alok Srivastva

Student Signature
Bhanu Sharma

Declaration

I, **Bhanu Sharma**, hereby solemnly declare that the project report titled **MERN Stack Employee Management System** submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Application is my original work.

This project has been carried out by me during the academic year **2024-2026** under the supervision of **Alok Srivastva**. The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.

All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.

Place: Chandigarh

Date: 05 July 2026

Student Signature

Bhanu Sharma

O24MCA112292

Acknowledgement

I would like to express my sincere gratitude to my guide and mentor, **Alok Srivastva**, for valuable guidance and support throughout this project. I also thank the faculty members of **Chandigarh University** for their encouragement and academic support.

I am thankful to all individuals who helped me complete this project successfully. Their support, feedback, and motivation helped me improve the quality of the application and documentation.

Abstract / Executive Summary

The MERN Stack Employee Management System is a web-based application designed to help companies manage employee information efficiently. The system provides secure user authentication, role-based access control, and CRUD operations for employee records such as name, contact details, department, job title, and employment status.

The backend is developed using Node.js and Express.js with MongoDB as the database. The frontend is developed using React. JWT authentication protects private routes and ensures that only authorized users can access employee data. Admin users can manage user roles, while employee data access is controlled according to user role.

The system includes dashboard summaries, employee search, filtering, sorting, pagination, CSV import with validation, CSV export, department summaries, profile management, activity logs, and Playwright testing. The project demonstrates database design, system analysis, REST API development, frontend integration, authentication, testing, and deployment readiness.

Table of Contents

Certificate	1
Declaration	2
Acknowledgement	3
Abstract / Executive Summary	4
Chapter 1: Introduction	5
Chapter 2: System Study	6
Chapter 3: System Analysis	7
Chapter 4: System Design	8
Chapter 5: System Implementation	9
Chapter 6: Testing	10
Chapter 7: Results And Discussion	11
Chapter 8: Conclusion And Future Scope	12
Chapter 9: References	13
Chapter 10: Appendices	14

Chapter 1: Introduction

Employee data management is an important operational requirement for every organization. Companies need to maintain accurate employee information such as personal contact details, department, job title, employment status, and account-level access. If this data is stored only in spreadsheets or paper records, it becomes difficult to search, update, secure, and audit.

The MERN Stack Employee Management System is a web application developed to solve this problem using MongoDB, Express.js, React, and Node.js. The project provides a secure interface for managing employee records and includes authentication, role-based authorization, search, filtering, CSV import/export, activity tracking, and a dashboard for summary information.

The application uses a company-style interface named BluePeak HR / PeopleDesk so that the project looks and behaves like a real workplace system. Instead of keeping all functionality on a single page, the application is divided into separate pages for dashboard, employees, employee details, add employee, edit employee, departments, CSV import, profile, users, and activity logs.

The main purpose of the project is to demonstrate a complete full-stack workflow: database design, backend API development, frontend implementation, authentication, authorization, testing, documentation, and deployment readiness.

Objectives

- To design a MongoDB database schema for storing user, employee, and activity log information.
- To develop RESTful APIs using Node.js and Express.js.
- To create a user-friendly React frontend for interacting with backend services.
- To implement JWT authentication and role-based access control.
- To provide employee CRUD, search, filtering, pagination, CSV import, and CSV export.
- To verify the application using build checks, audits, and Playwright browser testing.
- To prepare the application for cloud deployment using MongoDB Atlas, Render, and Vercel or Netlify.

Chapter 2: System Study

The existing manual method of employee record management usually depends on spreadsheets, documents, or scattered files. Such a method may work for a small number of records, but it becomes inefficient when employee data grows. Searching records, maintaining updated information, and controlling access become difficult.

A web-based employee management system improves the situation by centralizing the data in a database and providing a secure interface to authorized users. The proposed system supports structured employee records, quick search, department-wise view, role control, and activity logs.

The system also includes bulk import through CSV files. This is useful when a company already has employee data in spreadsheet format and wants to move it into the application without entering each employee manually.

Limitations Of Existing System

- Manual record keeping is time-consuming.
- Spreadsheet data can be duplicated or accidentally changed.
- Searching and filtering employee records is limited.
- Access control is weak in file-based systems.
- There is no activity history for tracking changes.
- Bulk operations and reporting are difficult.

Chapter 3: System Analysis

System analysis identifies the functional and non-functional requirements of the proposed application. The project is designed for users who need to manage employee records securely through a browser-based interface.

The system has two major roles: admin and user. Admin users can manage all employee records and user roles. Normal users can manage records created by them. This role-based design improves privacy and prevents unauthorized access.

The system also validates input data at both frontend and backend levels. CSV import validates required columns, email format, phone format, status values, and duplicate emails before importing records.

Functional Requirements

- User registration and login.
- JWT-based protected routes.
- Employee create, read, update, and delete operations.
- Employee details page.
- Employee search, filter, sort, and pagination.
- Bulk employee import using CSV with validation.
- Employee export to CSV.
- Department-wise employee summary.
- User profile management.
- Admin user role management.
- Activity log tracking.

Non-Functional Requirements

- The interface should be responsive and easy to use.
- Passwords should be stored securely using hashing.
- Environment variables should be used for secrets.
- The application should be deployable on cloud platforms.
- The codebase should be divided into readable modules.
- The system should handle reloads and invalid routes gracefully.

Chapter 4: System Design

The system follows a client-server architecture. The React frontend communicates with the Express backend using HTTP requests. The backend connects to MongoDB through Mongoose models. JWT tokens are used to authenticate users and protect private routes.

The frontend is divided into pages, components, services, context, and utilities. The backend is divided into routes, controllers, models, middleware, configuration, utilities, and data scripts. This separation improves maintainability and readability.

Important API Groups

- Authentication APIs: register, login, get current user, update profile.
- Employee APIs: list, create, details, update, delete, bulk import.
- User APIs: admin user list and role update.
- Activity APIs: recent activity logs.

Database Collections

Collection	Purpose	Important Fields
users	Stores application user accounts	name, email, password, role, timestamps
employees	Stores employee records	name, email, phone, department, jobTitle, status, createdBy, timestamps
activitylogs	Stores system activity history	action, message, entityType, entityId, user, timestamps

Chapter 5: System Implementation

The backend is implemented using Node.js and Express.js. MongoDB is connected through Mongoose. Passwords are hashed using bcryptjs, and JWT tokens are generated during registration and login. Middleware is used for authentication and role authorization.

The frontend is implemented using React and Vite. React Router provides navigation between pages. Axios is used for API communication. Authentication state is handled through a React context that stores the logged-in user and JWT token.

The employee module includes listing, searching, sorting, filtering, pagination, detail view, add form, edit form, delete operation, CSV export, and CSV import. The CSV import page validates data before sending it to the backend and shows row-level errors.

The activity module records important actions such as employee creation, employee update, employee deletion, CSV import, and user role changes. This helps the system look and behave like a real administrative tool.

Implemented Pages

- Login and register pages.
- Dashboard page with statistics, status overview, departments, recent employees, and activity.
- Employees page with search, filters, sorting, pagination, export, and actions.
- Employee details page.
- Add employee and edit employee pages.
- CSV import page with validation and sample CSV download.
- Departments page.
- Profile page.
- Admin users page.
- Activity log page.

Chapter 6: Testing

Testing was performed to verify that the application works correctly. The frontend production build was tested using Vite. Backend JavaScript files were checked for syntax errors. Client and server dependencies were audited using npm audit.

A Playwright smoke test was added to verify the main browser workflows. The test covers login, dashboard load, employees page, add employee, search, employee detail page reload, departments page, CSV import validation, edit page, delete operation, and cleanup of test records.

Testing Summary

Test Area	Test Method	Result
Frontend Build	npm run build in client	Passed
Dependency Audit	npm audit for client and server	0 vulnerabilities
Authentication	Playwright login flow	Passed
Employee CRUD	Playwright add/edit/delete flow	Passed
Search And Details	Playwright search and details reload	Passed
CSV Import	Valid and invalid row validation	Passed
Backend APIs	Manual API health and login checks	Passed

Chapter 7: Results And Discussion

The project successfully implements a full-stack employee management system using the MERN stack. The system provides secure login, employee record management, search, filtering, pagination, CSV import/export, department summary, user profile, admin user management, and activity tracking.

The application is structured in a way that makes it suitable for academic demonstration and future extension. The user interface is not a single-page demo screen; instead, it is divided into meaningful modules with separate URLs.

The final result is a working local application with GitHub source code, documentation, milestone reports, testing support, and deployment instructions.

Major Outcomes

- Centralized employee record management.
- Secure authentication and authorization.
- Improved usability with separate pages and a collapsible sidebar.
- Bulk CSV import with validation.
- Activity logs for administrative tracking.
- Testing and documentation support for project submission.

Chapter 8: Conclusion And Future Scope

The MERN Stack Employee Management System meets the major objectives of the project. It demonstrates database design, RESTful API development, frontend implementation, authentication, authorization, testing, and deployment readiness. The system can be used by a company to manage employee information in a structured and secure way.

The project helped demonstrate full-stack development skills using modern web technologies. It also shows how frontend and backend modules can work together through APIs and how security can be added using JWT authentication.

Future Scope

- Add attendance management.
- Add salary and payroll module.
- Add employee profile image upload.
- Add password reset through email.
- Add advanced reports and charts.
- Add audit log filters and export.
- Add Docker support for deployment.
- Deploy the final application on cloud infrastructure.

Chapter 9: References

1. MongoDB Documentation, <https://www.mongodb.com/docs/>
2. MongoDB Atlas Documentation, <https://www.mongodb.com/docs/atlas/>
3. Mongoose Documentation, <https://mongoosejs.com/docs/>
4. Express.js Documentation, <https://expressjs.com/>
5. Node.js Documentation, <https://nodejs.org/docs/>
6. React Documentation, <https://react.dev/>
7. React Router Documentation, <https://reactrouter.com/>
8. Vite Documentation, <https://vite.dev/>
9. JSON Web Token Introduction, <https://jwt.io/introduction>
10. bcryptjs Package Documentation, <https://www.npmjs.com/package/bcryptjs>
11. Axios Documentation, <https://axios-http.com/>
12. Playwright Documentation, <https://playwright.dev/>
13. Render Documentation, <https://render.com/docs>
14. Vercel Documentation, <https://vercel.com/docs>
15. Netlify Documentation, <https://docs.netlify.com/>
16. MDN Web Docs - JavaScript, <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
17. MDN Web Docs - Fetch and HTTP Concepts, <https://developer.mozilla.org/>
18. OWASP Authentication Cheat Sheet, <https://cheatsheetseries.owasp.org/>
19. npm Documentation, <https://docs.npmjs.com/>
20. GitHub Docs, <https://docs.github.com/>

Chapter 10: Appendices

Appendix A: Source Code Repository

The complete source code is maintained in the GitHub repository: <https://github.com/nextlvlpro/mern-employee-app>

Appendix B: Important Local URLs

Frontend	http://localhost:5173
Backend Health	http://localhost:5000/api/health

Appendix C: Important Commands

Install dependencies	<code>npm run install-all</code>
Start development servers	<code>npm run dev</code>
Stop development servers	<code>npm run stop:dev</code>
Run Playwright smoke test	<code>npm run test:e2e</code>

Appendix D: Screenshots To Attach

- Login page and dashboard.
- Employee list with search, filters, and pagination.
- Add employee, edit employee, and employee details pages.
- CSV import page with valid preview and validation errors.
- Departments, activity, profile, and admin users pages.
- MongoDB collections and GitHub repository.